

**ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО»
(Университет ИТМО)**

Факультет программной инженерии и компьютерных технологий

**Лабораторная работа №6
По предмету: “Вычислительная математика”
Вариант № 11**

Выполнил: Федоров Д. А.

Р3206

Проверил: Зинченко А. А.

г. Санкт-Петербург 2026 г.

СОДЕРЖАНИЕ

ЗАДАНИЕ 3

1. 3

2. 3

3. 3

4. 4

Исходный код программы:..... 4

Результаты работы программы:..... 5

Заключение:..... 7

ЗАДАНИЕ:

1. Порядок выполнения работы

2. В программе численные методы решения обыкновенных дифференциальных уравнений (ОДУ) должен быть реализован в виде отдельного класса /метода/функции;
3. Пользователь выбирает ОДУ вида $y' = f(x, y)$ (не менее трех уравнений), из тех, которые предлагает программа;
4. Предусмотреть ввод исходных данных с клавиатуры: начальные условия $y_0 = y(x_0)$, интервал дифференцирования $[x_0, x_n]$, шаг h , точность ε ;
5. Для исследования использовать одношаговые методы и многошаговые методы (см. табл.1);
6. Составить таблицу приближенных значений интеграла дифференциального уравнения, удовлетворяющего начальным условиям, для всех методов, реализуемых в программе;
7. Для оценки точности одношаговых методов использовать правило Рунге;
8. Для оценки точности многошаговых методов использовать точное решение задачи: $\varepsilon = \max_{0 \leq i \leq n} |y_{i\text{точн}} - y_i|$;
9. Построить графики точного решения и полученного приближенного решения (разными цветами);
10. Программа должна быть протестирована при различных наборах данных, в том числе и некорректных.
11. Проанализировать результаты работы программы.

Одношаговые методы:

1. Усовершенствованный метод Эйлера,
2. Метод Рунге-Кутты 4- го порядка.

Многошаговые методы (методы предиктор-корректор):

4. Адамса,

Программная реализация задачи:

Листинг программы:

```
import numpy as np

import matplotlib.pyplot as plt

ODES = {

    1: {'name': "y' = y", 'f': lambda x, y: y, 'exact': lambda x: np.exp(x)},

    2: {'name': "y' = 2x + y", 'f': lambda x, y: 2*x + y, 'exact': lambda x: 3*np.exp(x) - 2*x - 2},

    3: {'name': "y' = -2xy", 'f': lambda x, y: -2*x*y, 'exact': lambda x: np.exp(-x**2)}

}
```

```

def euler_modified(f, x0, y0, xn, h):

    x = np.arange(x0, xn + h, h)

    if x[-1] < xn:

        x = np.append(x, xn)

    n, y = len(x) - 1, np.zeros(len(x))

    y[0] = y0

    for i in range(n):

        y_pred = y[i] + h * f(x[i], y[i])

        y[i+1] = y[i] + h/2 * (f(x[i], y[i]) + f(x[i+1], y_pred))

    return x, y

```

```

def runge_kutta_4(f, x0, y0, xn, h):

    x = np.arange(x0, xn + h, h)

    if x[-1] < xn:

        x = np.append(x, xn)

    n, y = len(x) - 1, np.zeros(len(x))

    y[0] = y0

    for i in range(n):

        k1 = h * f(x[i], y[i])

        k2 = h * f(x[i] + h/2, y[i] + k1/2)

        k3 = h * f(x[i] + h/2, y[i] + k2/2)

        k4 = h * f(x[i] + h, y[i] + k3)

        y[i+1] = y[i] + (k1 + 2*k2 + 2*k3 + k4)/6

    return x, y

```

```

def adams_predictor_corrector(f, x0, y0, xn, h, max_iter=10, tol=1e-10):

    x = np.arange(x0, xn + h, h)

```

```

if x[-1] < xn:
    x = np.append(x, xn)
n, y = len(x) - 1, np.zeros(len(x))
y[0] = y0

x_start, y_start = runge_kutta_4(f, x0, y0, x0 + 3*h, h)
y[:4] = y_start[:4]

f_vals = np.zeros(n + 1)
for i in range(4):
    f_vals[i] = f(x[i], y[i])

for i in range(3, n):
    y_pred = y[i] + h/24 * (55*f_vals[i] - 59*f_vals[i-1] + 37*f_vals[i-2] - 9*f_vals[i-3])

    y_new = y_pred
    for _ in range(max_iter):
        f_next = f(x[i+1], y_new)
        y_corr = y[i] + h/24 * (9*f_next + 19*f_vals[i] - 5*f_vals[i-1] + f_vals[i-2])

        if abs(y_corr - y_new) < tol:
            y_new = y_corr
            break
        y_new = y_corr

    y[i+1] = y_new
    f_vals[i+1] = f(x[i+1], y[i+1])

return x, y

```

```
def choose_ode():  
    print("доступные ОДУ:")  
    for k, ode in ODES.items():  
        print(f" {k}. {ode['name']}")  
    while True:  
        line = input("\nвыберите уравнение(1-3): ").strip()  
        if line.isdigit() and int(line) in ODES:  
            return int(line)  
        print("Ошибка: введите число от 1 до 3")
```

```
def read_float(prompt):  
    while True:  
        line = input(prompt).strip().replace(',', '.')  
        if line:  
            try:  
                return float(line)  
            except:  
                print("Ошибка: введите число")  
        else:  
            print("Ошибка: поле не может быть пустым")
```

```
def read_positive(prompt):  
    while True:  
        v = read_float(prompt)  
        if v > 0:  
            return v  
        print(f"Ошибка: значение должно быть >0")
```

```
def plot_results(x_e, y_e, x_r, y_r, x_a, y_a, exact, h, eps):
```

```
    plt.figure(figsize=(12,5))
```

```
    plt.plot(x_e, y_e, '--', label='Эйлер')
```

```
    plt.plot(x_r, y_r, '-.', label='Рунге-Кутта4')
```

```
    plt.plot(x_a, y_a, ':', label='Адамс')
```

```
    if exact is not None:
```

```
        plt.plot(x_a, exact, 'k-', label='Точное')
```

```
    plt.xlabel('x')
```

```
    plt.ylabel('y')
```

```
    plt.legend()
```

```
    plt.grid(True)
```

```
    plt.title(f'Численное решение ОДУ (h={h},  $\epsilon$ ={eps})')
```

```
    plt.tight_layout()
```

```
    plt.show()
```

```
def main():
```

```
    ode = ODES[choose_ode()]
```

```
    print(f"\nВыбрано: {ode['name']}")
```

```
    print("\nВведите параметры:")
```

```
    x0 = read_float(" x0:")
```

```
    y0 = read_float(f" y({x0}):")
```

```
    xn = read_float(" xn:")
```

```
    while xn <= x0:
```

```
        xn = read_float(f" xn должно быть > {x0}:")
```

```
    h = read_positive(" h (>0):")
```

```

eps = read_positive(" ε (>0):")

x_e, y_e = euler_modified(ode['f'], x0, y0, xn, h)
x_r, y_r = runge_kutta_4(ode['f'], x0, y0, xn, h)
x_a, y_a = adams_predictor_corrector(ode['f'], x0, y0, xn, h)

exact = np.array([ode['exact'](x) for x in x_a]) if ode['exact'] is not None else None

print(f"{'x':^10} | {'Эйлер':^25} | {'Рунге-Кутта4':^25} | {'Адамс':^25} | {'Точное':^15}")
for i, x in enumerate(x_a):
    if exact is not None:
        print(f"{'x':^10.4f} | {y_e[i] if i<len(y_e) else 0:^25.6f} | {y_r[i]:^25.6f} | {y_a[i]:^25.6f} | {exact[i]:^15.6f}")
    else:
        print(f"{'x':^10.4f} | {y_e[i] if i<len(y_e) else 0:^25.6f} | {y_r[i]:^25.6f} | {y_a[i]:^25.6f}")

if exact is not None:
    err_e = np.max(np.abs(exact[:len(y_e)] - y_e))
    err_r = np.max(np.abs(exact[:len(y_r)] - y_r))
    err_a = np.max(np.abs(exact - y_a))

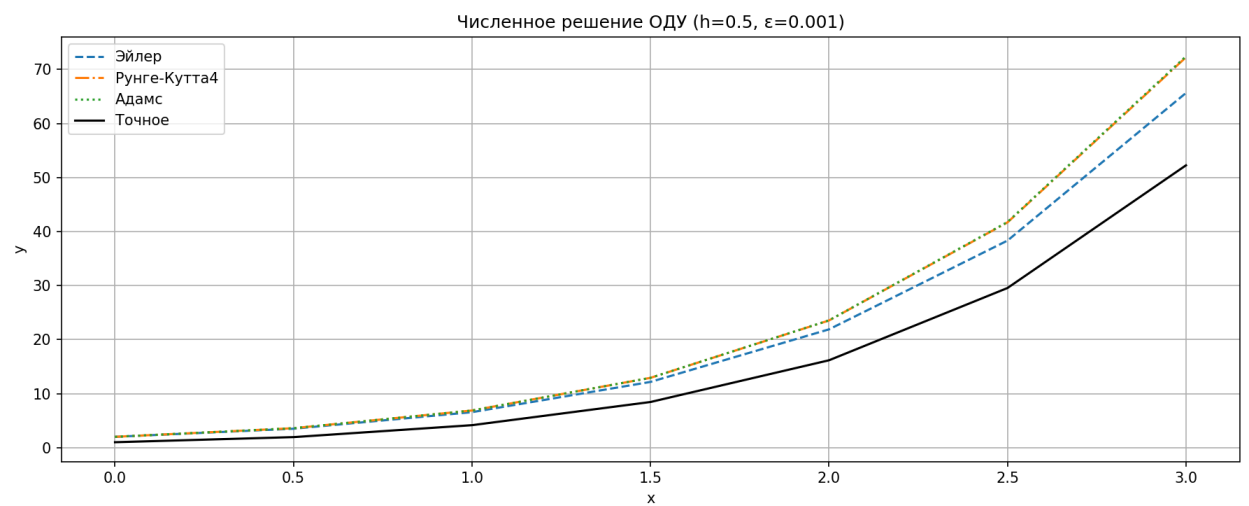
    print(f"\nПогрешность:\n Эйлер: {err_e:.6e}\n Рунге-Кутта: {err_r:.6e}\n Адамс: {err_a:.6e}")

plot_results(x_e, y_e, x_r, y_r, x_a, y_a, exact, h, eps)

if __name__ == "__main__":
    main()

```


Результаты работы программы:



```
доступные ОДУ:
1. y' = y
2. y' = 2x + y
3. y' = -2xy

выберите уравнение(1-3): 2

Выбрано: y' = 2x + y

Введите параметры:
x0:0
y(0.0):2
xp:3
h (>0):0.5
ε (>0):0.001

x          Эйлер          |          Рунге-Кутта4          |          Адамс          |          Точное
0.0000    2.000000        |          2.000000          |          2.000000        |          1.000000
0.5000    3.500000        |          3.593750          |          3.593750        |          1.946164
1.0000    6.562500        |          6.869385          |          6.869385        |          4.154845
1.5000    12.164062       |          12.917501         |          12.917501       |          8.445067
2.0000    21.891602       |          23.535881         |          23.535881       |          16.167168
2.5000    38.323853       |          41.688054         |          41.758839       |          29.547482
3.0000    65.651260       |          72.259215         |          72.435261       |          52.256611

Погрешность:
Эйлер: 1.339465e+01
Рунге-Кутта: 2.000260e+01
Адамс: 2.017865e+01
PS D:\vichmat>
```

Вывод: В данной лабораторной работе я изучил численные методы решения обыкновенных дифференциальных уравнений.